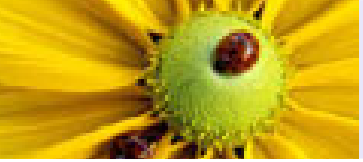
Linguaggi di Programmazione I – Lezione 1

Prof. Marco Faella
http://wpage.unina.it/mfaella

Materiale didattico elaborato con i Proff. Sette e Bonatti

8 marzo 2023



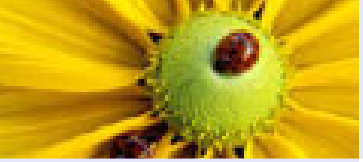
Panoramica della lezione

Introduzione al corso

Linguaggi (di programmazione)

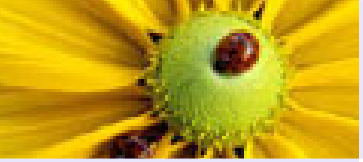
Macchine astratte

Paradigmi computazionali



Introduzione al corso
Obbiettivi specifici
Obbiettivi generali
Scheduling
Altre informazioni
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali

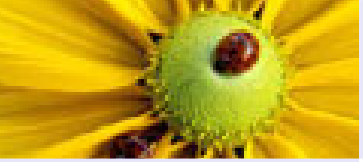
Introduzione al corso



Obbiettivi specifici

Introduzione al corso
Obbiettivi specifici
Obbiettivi generali
Scheduling
Altre informazioni
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali

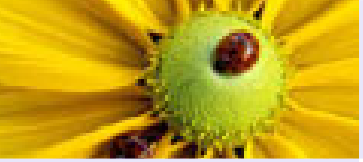
- Mettere lo studente in grado di imparare velocemente un nuovo linguaggio di programmazione
  - mediante astrazione delle *caratteristiche costituenti* dei linguaggi
  - cambiano più di rado di quanto vengano introdotti nuovi linguaggi
- Spiegare le differenze tra i vari paradigmi di programmazione - imperativo, ad oggetti, funzionale, logico - e sul loro impatto sullo stile di soluzione dei problemi.
- Mostrare cenni sui criteri di progetto e le tecniche d'implementazione dei linguaggi di programmazione
- Esempi focalizzati su Java, ML, Prolog. Ma anche C, Pascal, e Python.



Obbiettivi generali

Introduzione al corso
Obbiettivi specifici
Obbiettivi generali
Scheduling
Altre informazioni
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali

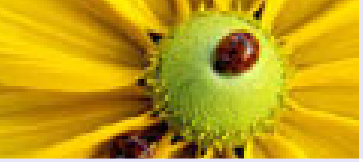
- Migliorare l'abilità nel risolvere i problemi usando meglio i linguaggi di programmazione.



Obbiettivi generali

Introduzione al corso
Obbiettivi specifici
Obbiettivi generali
Scheduling
Altre informazioni
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali

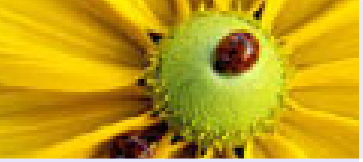
- Migliorare l'abilità nel risolvere i problemi usando meglio i linguaggi di programmazione.
- Imparare a scegliere più intelligentemente, in dipendenza del problema, il linguaggio di programmazione.



Obbiettivi generali

Introduzione al corso
Obbiettivi specifici
Obbiettivi generali
Scheduling
Altre informazioni
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali

- Migliorare l'abilità nel risolvere i problemi usando meglio i linguaggi di programmazione.
- Imparare a scegliere più intelligentemente, in dipendenza del problema, il linguaggio di programmazione.
- Aumentare la capacità di imparare linguaggi di programmazione (che sono TANTI!)

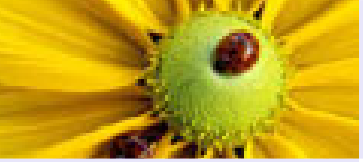


Scheduling

Introduzione al corso
Obbiettivi specifici
Obbiettivi generali
Scheduling
Altre informazioni
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali

- Parte prima

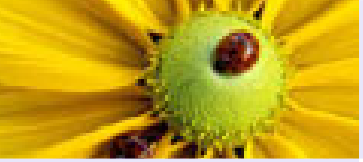
1. Storia e concetti di base
2. Primi cenni ai paradigmi dei linguaggi di programmazione.
3. Il modello imperativo.



Scheduling

Introduzione al corso
Obbiettivi specifici
Obbiettivi generali
Scheduling
Altre informazioni
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali

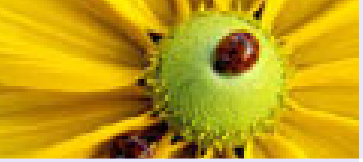
- Parte prima
  1. Storia e concetti di base
  2. Primi cenni ai paradigmi dei linguaggi di programmazione.
  3. Il modello imperativo.
- Parte seconda (modello object-oriented, Java)
  1. Studio dei costrutti più avanzati: qualificatori di classi, metodi e attributi; classi astratte; interfacce; template; classi interne; gestione degli errori: eccezioni.



Scheduling

Introduzione al corso
Obbiettivi specifici
Obbiettivi generali
Scheduling
Altre informazioni
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali

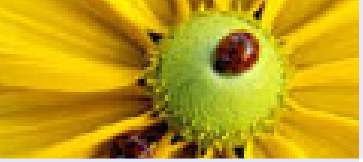
- Parte prima
  1. Storia e concetti di base
  2. Primi cenni ai paradigmi dei linguaggi di programmazione.
  3. Il modello imperativo.
- Parte seconda (modello object-oriented, Java)
  1. Studio dei costrutti più avanzati: qualificatori di classi, metodi e attributi; classi astratte; interfacce; template; classi interne; gestione degli errori: eccezioni.
- Parte terza (Linguaggi funzionali, ML)
  1. ML e costrutti funzionali: funzioni di ordine superiore; polimorfismo; tipi di dato astratti e interfacce; template; gestione degli errori: eccezioni; type inference.



Scheduling

Introduzione al corso
Obbiettivi specifici
Obbiettivi generali
Scheduling
Altre informazioni
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali

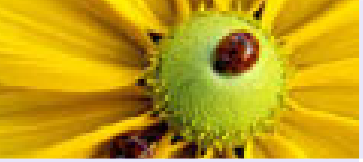
- Parte prima
  1. Storia e concetti di base
  2. Primi cenni ai paradigmi dei linguaggi di programmazione.
  3. Il modello imperativo.
- Parte seconda (modello object-oriented, Java)
  1. Studio dei costrutti più avanzati: qualificatori di classi, metodi e attributi; classi astratte; interfacce; template; classi interne; gestione degli errori: eccezioni.
- Parte terza (Linguaggi funzionali, ML)
  1. ML e costrutti funzionali: funzioni di ordine superiore; polimorfismo; tipi di dato astratti e interfacce; template; gestione degli errori: eccezioni; type inference.
- Parte quarta (Linguaggi logici, Prolog)
  1. Costrutti fondamentali: termini, predicati e regole.
  2. Tecniche di programmazione avanzate: invertibilità e nondeterminismo.



Altre informazioni

Introduzione al corso
Obbiettivi specifici
Obbiettivi generali
Scheduling
Altre informazioni
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali

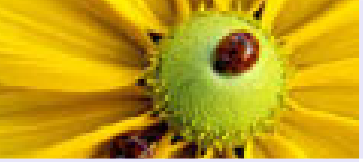
- *Libri di testo*:
  - Gabbrielli - Martini. *Linguaggi di programmazione: Principi e paradigmi*
  - Dershem - Jipping. *Programming languages: structures and models*.
  - Slide e dispense fornite dal docente



Altre informazioni

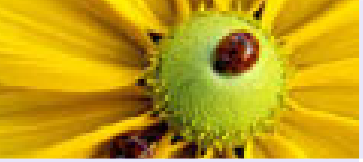
Introduzione al corso
Obbiettivi specifici
Obbiettivi generali
Scheduling
Altre informazioni
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali

- *Libri di testo*:
  - Gabbrielli - Martini. *Linguaggi di programmazione: Principi e paradigmi*
  - Dershem - Jipping. *Programming languages: structures and models*.
  - Slide e dispense fornite dal docente
- *Materiali*: Sul gruppo Teams e su http://wpage.unina.it/mfaella
- *Ricevimento studenti*: in presenza o in remoto nel gruppo Teams "Ricevimento Faella" (orario sulla homepage)



Introduzione al corso
Linguaggi (di programmazione)
Sono ~ 40?
Sono ~ 80?
Quanti sono?
Quanti sono?
Breve storia
Storia dei . . .
. . . concetti introdotti
Terminologia
Linguaggi completi
Macchine astratte
Paradigmi computazionali

Linguaggi (di programmazione)



Sono ~ 40?

Introduzione al corso

Linguaggi (di programmazione)

Sono ~ 40?

Sono ~ 80?

Quanti sono?

Quanti sono?

Breve storia

Storia dei . . .

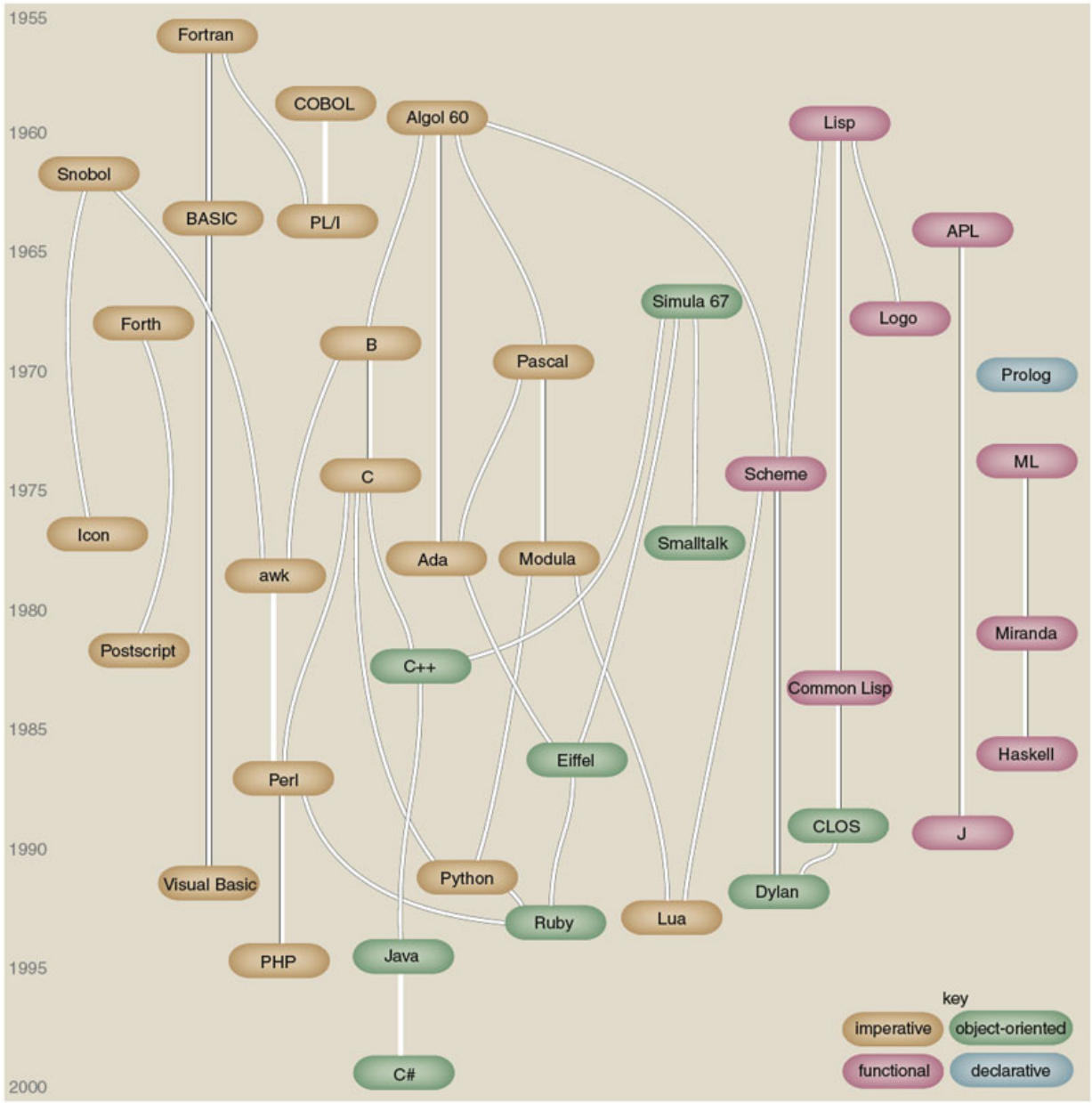
. . . concetti introdotti

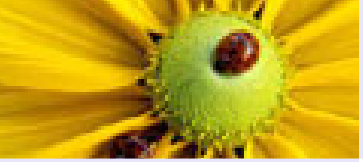
Terminologia

Linguaggi completi

Macchine astratte

Paradigmi computazionali





Sono ~ 80?

Introduzione al corso

Linguaggi (di programmazione)

Sono ~ 40?

Sono ~ 80?

Quanti sono?

Quanti sono?

Breve storia

Storia dei . . .

. . . concetti introdotti

Terminologia

Linguaggi completi

Macchine astratte

Paradigmi computazionali

Mother Tongues

Tracing the roots of computer languages through the ages

Just like half of the world's spoken tongues, most of the 2,300-plus computer programming languages are either endangered or extinct. As powerhouses C/C++, Visual Basic, Cobol, Java, and other modern source codes dominate our systems, hundreds of older languages are running out of life.

An ad hoc collection of engineers – electronic lexicographers, if you will – aim to save, or at least document, the lingo of classic software. They're combing the globe's 9 million developers in search of coders still fluent in these nearly forgotten lingua francas. Among the most endangered are Ada, APL, B (the predecessor of C), Lisp, Oberon, Smalltalk, and Simula.

Code-raker Grady Booch, Rational Software's chief scientist, is working with the Computer History Museum in Silicon Valley to record and, in some cases, maintain languages by writing new compilers so our ever-changing hardware can grok the code. Why bother? "They tell us about the state of software practice, the minds of their inventors, and the technical, social, and economic forces that shaped history at the time," Booch explains. "They'll provide the raw material for software archaeologists, historians, and developers to learn what worked, what was brilliant, and what was an utter failure." Here's a peek at the strongest branches of programming's family tree. For a nearly exhaustive rundown, check out the Language List at *www.informatik.uni-freiburg.de/Java/misc/lang_list.html*. – **Michael Menduno**

Key

1954 **Year Introduced**

Active: thousands of users

Protected: taught at universities; compilers available

Endangered: usage dropping off

Extinct: no known active users or up-to-date compilers

Lineage continues

1954 1955 1956 1957 1958 1959 1960 1961 1962 1963 1964 1965 1966 1967 1968 1969 1970 1971 1972 1973 1974 1975 1976 1977 1978 1979 1980 1981 1982 1983 1984 1985 1986 1987 1988 1989 1990 1991 1992 1993 1994 1995 1996 1997 1998 1999 2000 2001

Fortran — Once one of the most widely used languages in science and engineering.

Flow-Matic — Originally B-0. One of the first English-like data-processing languages.

Cobol — The culprit behind many Y2K bugs. Also known as Common Business-Oriented Language.

Cobol 61

Fortran IV — Created for the IBM 7090/94.

ANSI Cobol 74

ANSI Cobol 85

Fortran 90 ISO/IEC

OO Cobol

IPL — Possibly the first list-processing language; used in AI guru Ed Feigenbaum's PhD thesis.

Lisp — Invented by John McCarthy at MIT. Lisp has an unusual syntax made up of lots of nested parentheses. Still popular with AI researchers.

George Radin's attempt to combine the best features of Fortran, Cobol, and Algol 60.

Logo — A Lisp-like language designed by MIT researcher Seymour Papert to make programming accessible to children through graphics and simple syntax.

Scheme — A Lisp offshoot that's still used in introductory programming classes at universities.

Scheme 84

Scheme IEEE

Scheme R5RS

Haskell 1.0

Haskell 1.1

Object Logo

Clos — Common Lisp Object System

Common Lisp

ANSI Common Lisp

PL/1 — Algorithmic Language, designed as a portable language for scientific computations. Algol 68's complexity was either ahead of its time or rightfully doomed, depending on whom you ask.

ANSI PL/1

PL/M — Programming Language/Microcomputers

Self

Self 4.0

The version Xerox released to the public.

A simple object-oriented language.

NetRexx

Initially called Oak. Still one of the fastest-growing languages around, despite the standards feud between Sun and Microsoft. Somewhat like C++, Java allows for "write once, run anywhere" portability across the Net.

Smalltalk — The first language to show off the power of object-oriented code. Developed by Software Concepts Group, Xerox PARC, led by Alan Key.

Smalltalk-80

Algol 58 — The hopid-for Esperanto of the computing world. Designed by an international Zurich-based committee as a universal language, it was one of the first attempts at making software more portable. Initially called International Algebraic Language.

Algol 68

Rex 1.00

Rex 2.00 — Restructured Extended Executor

Object Rexx

Simula — Popular in Europe during the 70s, Simula introduced the now-standard concept of object-oriented, rather than procedural, programming. Designed by Kristen Nygaard and Ole-Johan Dahl.

Simula 67 — All-purpose version of Simula.

An object-oriented language.

A simple language (A = argument, B = basic value, C = ?).

Interactive object-oriented language.

Java

Java 2 (v1.3)

Prolog — Programmation en Logique for natural-language processing. Popular for AI programs.

Clu — Called Cluster, it's an object-oriented academic language created to teach rigid engineering skills.

Macro and text-processing language used by Unix admins to manipulate large config files programmatically, rather than editing them by hand.

Objective-C

Eiffel

Eiffel 3

Microsoft's answer to Java, it is a key component of the Microsoft .Net platform for Web services.

ABC

Sather 0.1

Ruby

A hardware description language.

New AWK, a pattern scanning and processing language.

AWK

NAWK

Python

Python 1.5

A popular language among Web site builders, it includes features missing from Perl.

ECMA Script

Possibly the most common language today. Adds object-oriented features to C.

Tool Command Language, "tickle." The duct tape of programming (a scripting language for patching together different languages).

Cmm

C--, a scripting language.

JavaScript

JScript

Microsoft's version of JavaScript. Can't they share anything? ;-)

C#

C with Classes

C++

Internet C++

Concurrent C

Tcl

Tcl/Tk

Tcl/Tk 8.1

One of the most widely deployed languages today. Windows and Unix OSes are written mostly in C and its descendants.

BCPL — Basic Combined Programming Language

B — Used primarily for non-numeric programming.

C

ANSI C (C89)

ISO C (C95)

Updates C++ for the Net with a Java-like virtual machine, so code can run on any kind of computer.

ADA

ADA 83 — The US Department of Defense's effort to craft a standard object-oriented language for its work. Named after Ada Lovelace, arguably the world's first computer programmer, and created by Jean Ichbiah's team at Honeywell.

ADA 95

Modula — Modular Language

A teaching language named for French mathematician Blaise Pascal. Designed by legendary Swiss programmer Niklaus Wirth for simplicity, in reaction to the complexity of Algol 68.

Modula 3

Found in millions of Web pages. Originally dubbed LiveScript, it was renamed by Netscape marketers who licensed the name to ride Java's buzz. It has little in common with that language.

Modula 2 ISO

Object-oriented Pascal, designed for simplicity.

Object Pascal

Oberon — Another creation of Niklaus Wirth. Later modified by Robert Griesemer for numerical apps on supercomputers.

Delphi

Pascal

Csh — C-Shell, a scripting language and command-shell interpreter. Written by C programmers to make Unix command lines more like C's.

The Swiss Army Knife of programming (a.k.a Practical Extraction and Report Language), used for patching together different languages. Spawned a quasi-literary culture that writes Perl haiku.

Perl 1.000

Perl 4.000

sh

Ksh — Frame-based language.

Bash — The kitchen sink of command-line programming features for Unix, based on the original Bourne shell (a.k.a Bourne Again Shell).

String-processing language for text and formula manipulation, common in text processors. Invented by David Ferber, Ralph Griswold, and Ivan Polonsky at Bell Labs.

Programmer's easy street – with this descendant of Snobol, there's no need to know the underlying OS.

A general-purpose language known for its beauty and grace. Designed by Ralph Griswold to be successor to Snobol4.

King of the one-liners, also referred to as A Programming Language. An algebra-like language with a special set of characters for writing complex programs as short formulas. Designed by Kenneth Iverson at Harvard.

Snobol

Snobol4

Icon

ML

Standard ML, a general-purpose language.

SML

SML 90

SML 97

Meta Language

Page-description language for printers and graphics systems; 75 percent of all commercial documents are produced on PostScript printers.

Caml

Caml 2-6.1

Objective Caml

O Caml 2

O Caml 3.00

Word-based language first used to guide the National Radio Astronomy Observatory telescope at Kitt Peak in Arizona.

Forth

PostScript

PostScript Level 2

Categorical Abstract Machine Language

J

Although mocked by "real" programmers for its limited abilities, Basic has outlived many more advanced languages, as well as the RadioShack TRS-80 computers that made it a household word. Stands for Beginner's All-Purpose Symbolic Instruction Code.

APL

APL 96

Basic

Microsoft Basic

MS Basic 2.0

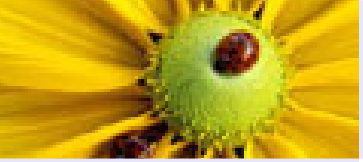
Visual Basic — The Rodney Dangerfield of programming languages. Popular for building Web sites with Microsoft's Visual Studio tools.

Survival of the Fittest

Reasons a language endures, with examples of some classic tongues

- **Appeals to a wide audience** C (bolstered by the popularity of Unix)
- **Gets a job done** Cobol (designed for business-report writing)
- **Delivers new functionality** Java (runs on any hardware platform)
- **Fills a niche** Mathematica (speeds up complex computations)
- **Offers a modicum of elegance** Icon (has friendly, line-oriented syntax)
- **Has a powerful user base or backer** C# (developed by Microsoft for .Net)
- **Has a charismatic leader** Perl (programmer-author Larry Wall)

Sources: Paul Boutin; Brent Hailpern, associate director of computer science at IBM Research; The Retrocomputing Museum; Todd Proebsting, senior researcher at Microsoft; Gio Wiederhold, computer scientist, Stanford University



Quanti sono?

Introduzione al corso

Linguaggi (di programmazione)

Sono ~ 40?

Sono ~ 80?

Quanti sono?

Quanti sono?

Breve storia

Storia dei . . .

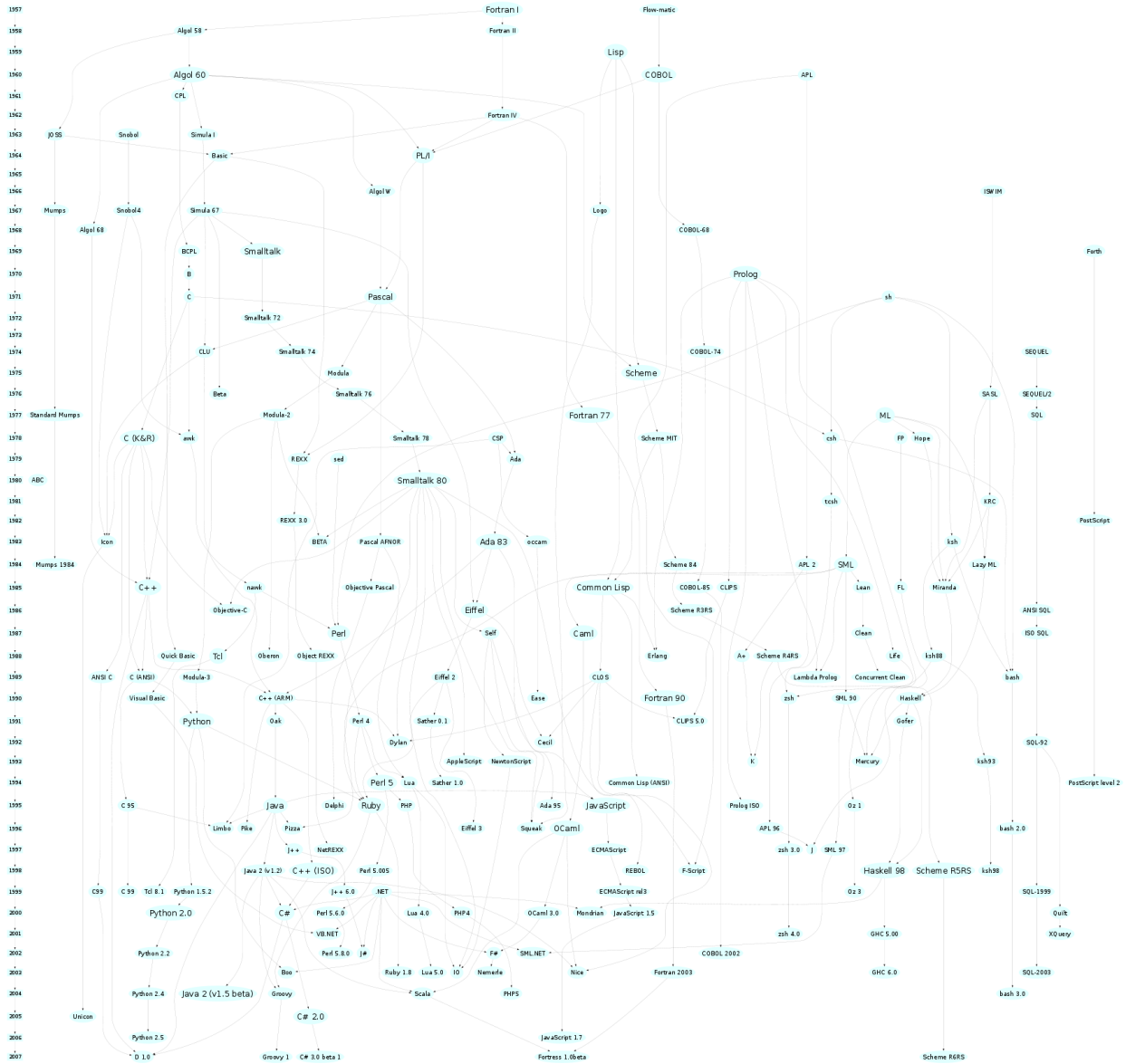
. . . concetti introdotti

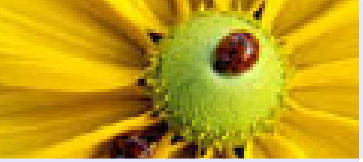
Terminologia

Linguaggi completi

Macchine astratte

Paradigmi computazionali

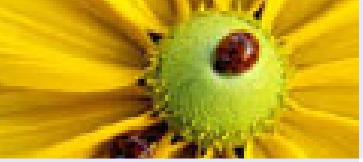




Quanti sono?

Introduzione al corso
Linguaggi (di programmazione)
Sono ~ 40?
Sono ~ 80?
Quanti sono?
Quanti sono?
Breve storia
Storia dei . . .
. . . concetti introdotti
Terminologia
Linguaggi completi
Macchine astratte
Paradigmi computazionali

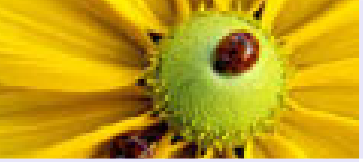
- C'è chi dice addirittura qualche migliaio
  - Come orientarsi?
  - Come usarli *bene*?
  - Come apprendere in fretta quelli nuovi?
- Occorre comprensione astratta delle caratteristiche dei linguaggi per coglierne somiglianze/differenze
- e per comprendere lo scopo di ciascun costrutto (ovvero i principi del *language design*)



Breve storia

Introduzione al corso
Linguaggi (di programmazione)
Sono ~ 40?
Sono ~ 80?
Quanti sono?
Quanti sono?
Breve storia
Storia dei . . .
. . . concetti introdotti
Terminologia
Linguaggi completi
Macchine astratte
Paradigmi computazionali

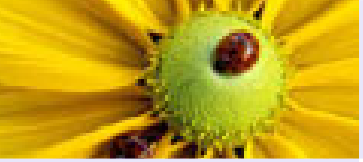
1954	FORTRAN (FORmula TRANslation)



Breve storia

Introduzione al corso
Linguaggi (di programmazione)
Sono ~ 40?
Sono ~ 80?
Quanti sono?
Quanti sono?
Breve storia
Storia dei . . .
. . . concetti introdotti
Terminologia
Linguaggi completi
Macchine astratte
Paradigmi computazionali

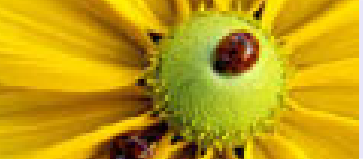
1954	FORTRAN (FORmula TRANslation)
1960	COBOL (Common Business Oriented Language) ALGOL 60 (Algorithmic Oriented Language) PL/1 (Programming Language 1) Simula 67 ALGOL 68 PASCAL LISP (LISt Processing) APL BASIC



Breve storia

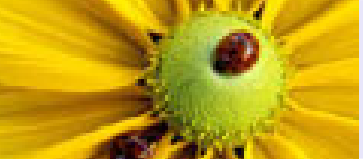
Introduzione al corso
Linguaggi (di programmazione)
Sono ~ 40?
Sono ~ 80?
Quanti sono?
Quanti sono?
Breve storia
Storia dei . . .
. . . concetti introdotti
Terminologia
Linguaggi completi
Macchine astratte
Paradigmi computazionali

1954	FORTRAN (FORmula TRANslation)
1960	COBOL (Common Business Oriented Language) ALGOL 60 (Algorithmic Oriented Language) PL/1 (Programming Language 1) Simula 67 ALGOL 68 PASCAL LISP (LISt Processing) APL BASIC
1970/80	PROLOG SMALLTALK C MODULA/2 ADA



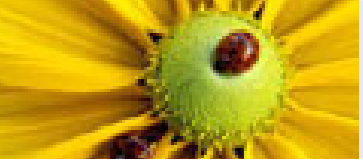
Storia dei concetti introdotti dai progenitori dell'oggi

Fortran: nato per manipolazione algebrica; introduce: variabili, statement di assegnazione, concetto di tipo, subroutine, iterazione e statement condizionali, go to, formati di input e output.
Gestione solo statica della memoria, no ricorsione, no strutture dinamiche, no tipi definiti da utente.



Storia dei concetti introdotti dai progenitori dell'oggi

Fortran: nato per manipolazione algebrica; introduce: variabili, statement di assegnazione, concetto di tipo, subroutine, iterazione e statement condizionali, go to, formati di input e output.
Gestione solo statica della memoria, no ricorsione, no strutture dinamiche, no tipi definiti da utente.
Cobol: indipendenza dalla macchina e statement "English like". Orientato ai database. Introduce il record.

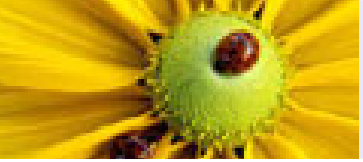


Storia dei concetti introdotti dai progenitori dell'oggi

Fortran: nato per manipolazione algebrica; introduce: variabili, statement di assegnazione, concetto di tipo, subroutine, iterazione e statement condizionali, go to, formati di input e output.
Gestione solo statica della memoria, no ricorsione, no strutture dinamiche, no tipi definiti da utente.

Cobol: indipendenza dalla macchina e statement "English like". Orientato ai database. Introduce il record.

Algol60: indipendenza dalla macchina e definizione mediante grammatica *(Backus-Naur form)*, strutture a blocco, supporto generale dell'iterazione e ricorsione.



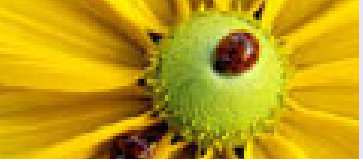
Storia dei concetti introdotti dai progenitori dell'oggi

Fortran: nato per manipolazione algebrica; introduce: variabili, statement di assegnazione, concetto di tipo, subroutine, iterazione e statement condizionali, go to, formati di input e output.
Gestione solo statica della memoria, no ricorsione, no strutture dinamiche, no tipi definiti da utente.

Cobol: indipendenza dalla macchina e statement "English like". Orientato ai database. Introduce il record.

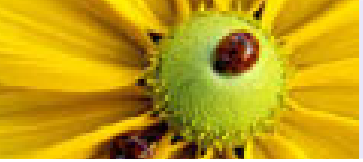
Algol60: indipendenza dalla macchina e definizione mediante grammatica *(Backus-Naur form)*, strutture a blocco, supporto generale dell'iterazione e ricorsione.

Lisp: primo vero linguaggio di manipolazione simbolica, paradigma funzionale, non c'è lo statement di assegnazione, e quindi concettualmente non c'è "il valore" ovvero l'idea di cambiare lo stato della memoria. Non c'è differenza concettuale fra funzione e dato: dipende dall'uso. La prima versione era essenzialmente non tipata.



Storia dei concetti introdotti dai progenitori dell'oggi

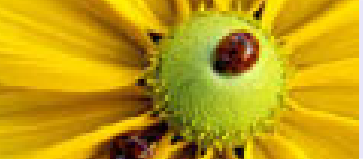
Prolog: primo (e principale) linguaggio di programmazione logica (paradigma logico). Tra le caratteristiche innovative: invertibilità, programmazione in stile nondeterministico *(generate and test)*. Essenzialmente non tipato; estensioni (tipi e altro) mediante metaprogrammazione.



Storia dei concetti introdotti dai progenitori dell'oggi

Prolog: primo (e principale) linguaggio di programmazione logica (paradigma logico). Tra le caratteristiche innovative: invertibilità, programmazione in stile nondeterministico *(generate and test)*. Essenzialmente non tipato; estensioni (tipi e altro) mediante metaprogrammazione.

Simula 67: classe come incapsulamento di dati e procedure, istanze delle classi (oggetti): anticipatorio del concetto di tipo di dato astratto implementato in Ada e Modula2, e del concetto moderno di classe di Smalltalk e C++.

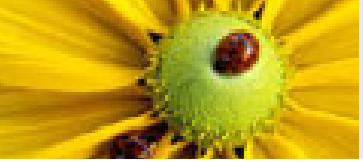


Storia dei concetti introdotti dai progenitori dell'oggi

Prolog: primo (e principale) linguaggio di programmazione logica (paradigma logico). Tra le caratteristiche innovative: invertibilità, programmazione in stile nondeterministico *(generate and test)*. Essenzialmente non tipato; estensioni (tipi e altro) mediante metaprogrammazione.

Simula 67: classe come incapsulamento di dati e procedure, istanze delle classi (oggetti): anticipatorio del concetto di tipo di dato astratto implementato in Ada e Modula2, e del concetto moderno di classe di Smalltalk e C++.

PL/1: abilità ad eseguire procedure specificate quando si verifica una condizione eccezionale; "multitasking", cioè specificazione di tasks che possono essere eseguiti in concorrenza.



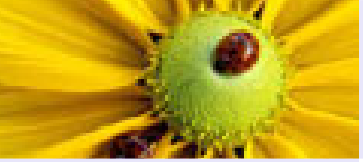
Storia dei concetti introdotti dai progenitori dell'oggi

Prolog: primo (e principale) linguaggio di programmazione logica (paradigma logico). Tra le caratteristiche innovative: invertibilità, programmazione in stile nondeterministico *(generate and test)*. Essenzialmente non tipato; estensioni (tipi e altro) mediante metaprogrammazione.

Simula 67: classe come incapsulamento di dati e procedure, istanze delle classi (oggetti): anticipatorio del concetto di tipo di dato astratto implementato in Ada e Modula2, e del concetto moderno di classe di Smalltalk e C++.

PL/1: abilità ad eseguire procedure specificate quando si verifica una condizione eccezionale; "multitasking", cioè specificazione di tasks che possono essere eseguiti in concorrenza.

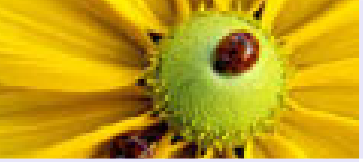
Pascal: programmazione strutturata, tipi di dato definiti da utente, ricchezza di strutture dati. Ma ancora niente encapsulation; si dovrà aspettare Modula.



Terminologia

Introduzione al corso
Linguaggi (di programmazione)
Sono ~ 40?
Sono ~ 80?
Quanti sono?
Quanti sono?
Breve storia
Storia dei . . .
. . . concetti introdotti
Terminologia
Linguaggi completi
Macchine astratte
Paradigmi computazionali

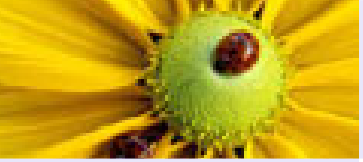
- **Linguaggio di programmazione**: è un linguaggio che è usato per esprimere (mediante un programma) un algoritmo con il quale un processore può risolvere un problema.



Terminologia

Introduzione al corso
Linguaggi (di programmazione)
Sono ~ 40?
Sono ~ 80?
Quanti sono?
Quanti sono?
Breve storia
Storia dei . . .
. . . concetti introdotti
Terminologia
Linguaggi completi
Macchine astratte
Paradigmi computazionali

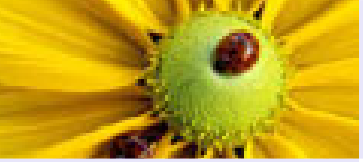
- **Linguaggio di programmazione**: è un linguaggio che è usato per esprimere (mediante un programma) un algoritmo con il quale un processore può risolvere un problema.
- **Programma**: è la codifica di un algoritmo in un linguaggio di programmazione.
- **Processore**: è la macchina (reale o virtuale) che eseguirà l'algoritmo descritto dal programma. Non si deve intendere come un singolo oggetto, ma come una *architettura di elaborazione*.



Terminologia

Introduzione al corso
Linguaggi (di programmazione)
Sono ~ 40?
Sono ~ 80?
Quanti sono?
Quanti sono?
Breve storia
Storia dei . . .
. . . concetti introdotti
Terminologia
Linguaggi completi
Macchine astratte
Paradigmi computazionali

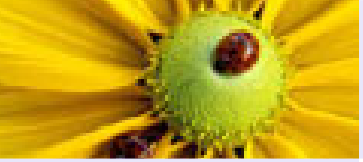
- **Linguaggio di programmazione**: è un linguaggio che è usato per esprimere (mediante un programma) un algoritmo con il quale un processore può risolvere un problema.
- **Programma**: è la codifica di un algoritmo in un linguaggio di programmazione.
- **Processore**: è la macchina (reale o virtuale) che eseguirà l'algoritmo descritto dal programma. Non si deve intendere come un singolo oggetto, ma come una *architettura di elaborazione*.



Linguaggi completi

Introduzione al corso
Linguaggi (di programmazione)
Sono ~ 40?
Sono ~ 80?
Quanti sono?
Quanti sono?
Breve storia
Storia dei . . .
. . . concetti introdotti
Terminologia
Linguaggi completi
Macchine astratte
Paradigmi computazionali

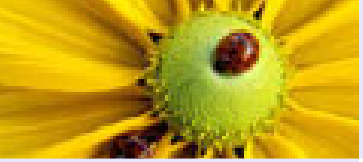
- È uso comune intendere come linguaggi di programmazione solo quelli *computazionalmente completi*, cioè solo quelli che possono esprimere *qualunque funzione calcolabile*
  - detti anche *general purpose*
  - tecnicamente: devono poter simulare qualunque *macchina di Turing*.



Linguaggi completi

Introduzione al corso
Linguaggi (di programmazione)
Sono ~ 40?
Sono ~ 80?
Quanti sono?
Quanti sono?
Breve storia
Storia dei . . .
. . . concetti introdotti
Terminologia
Linguaggi completi
Macchine astratte
Paradigmi computazionali

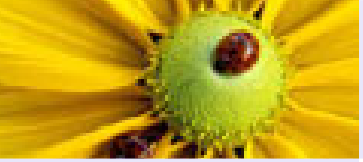
- È uso comune intendere come linguaggi di programmazione solo quelli *computazionalmente completi*, cioè solo quelli che possono esprimere *qualunque funzione calcolabile*
  - detti anche *general purpose*
  - tecnicamente: devono poter simulare qualunque *macchina di Turing*.
- Sono completi solo quelli che riescono ad esprimere anche programmi di cui non è decidibile la terminazione



Linguaggi completi

Introduzione al corso
Linguaggi (di programmazione)
Sono ~ 40?
Sono ~ 80?
Quanti sono?
Quanti sono?
Breve storia
Storia dei . . .
. . . concetti introdotti
Terminologia
Linguaggi completi
Macchine astratte
Paradigmi computazionali

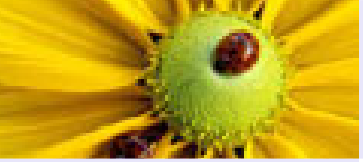
- È uso comune intendere come linguaggi di programmazione solo quelli *computazionalmente completi*, cioè solo quelli che possono esprimere *qualunque funzione calcolabile*
  - detti anche *general purpose*
  - tecnicamente: devono poter simulare qualunque *macchina di Turing*.
- Sono completi solo quelli che riescono ad esprimere anche programmi di cui non è decidibile la terminazione
- SQL non è un linguaggio completo (perché la terminazione dei programmi è sempre decidibile).



Linguaggi completi

Introduzione al corso
Linguaggi (di programmazione)
Sono ~ 40?
Sono ~ 80?
Quanti sono?
Quanti sono?
Breve storia
Storia dei . . .
. . . concetti introdotti
Terminologia
Linguaggi completi
Macchine astratte
Paradigmi computazionali

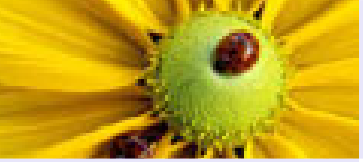
- È uso comune intendere come linguaggi di programmazione solo quelli *computazionalmente completi*, cioè solo quelli che possono esprimere *qualunque funzione calcolabile*
  - detti anche *general purpose*
  - tecnicamente: devono poter simulare qualunque *macchina di Turing*.
- Sono completi solo quelli che riescono ad esprimere anche programmi di cui non è decidibile la terminazione
- SQL non è un linguaggio completo (perché la terminazione dei programmi è sempre decidibile). È spesso immerso in linguaggi completi.



Linguaggi completi

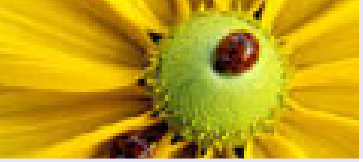
Introduzione al corso
Linguaggi (di programmazione)
Sono ~ 40?
Sono ~ 80?
Quanti sono?
Quanti sono?
Breve storia
Storia dei . . .
. . . concetti introdotti
Terminologia
Linguaggi completi
Macchine astratte
Paradigmi computazionali

- È uso comune intendere come linguaggi di programmazione solo quelli *computazionalmente completi*, cioè solo quelli che possono esprimere *qualunque funzione calcolabile*
  - detti anche *general purpose*
  - tecnicamente: devono poter simulare qualunque *macchina di Turing*.
- Sono completi solo quelli che riescono ad esprimere anche programmi di cui non è decidibile la terminazione
- SQL non è un linguaggio completo (perché la terminazione dei programmi è sempre decidibile). È spesso immerso in linguaggi completi.
- HTML non è un linguaggio completo (idem).
- Per mostrare la completezza di un linguaggio: usarlo per simulare arbitrarie macchine di Turing



Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

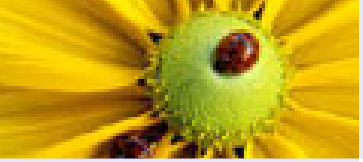
Macchine astratte



Definizione

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

Dato un linguaggio di programmazione L, una macchina astratta per L (in simboli, M_L) è un qualsiasi insieme di strutture dati e algoritmi che permettano di memorizzare ed eseguire programmi scritti in L.

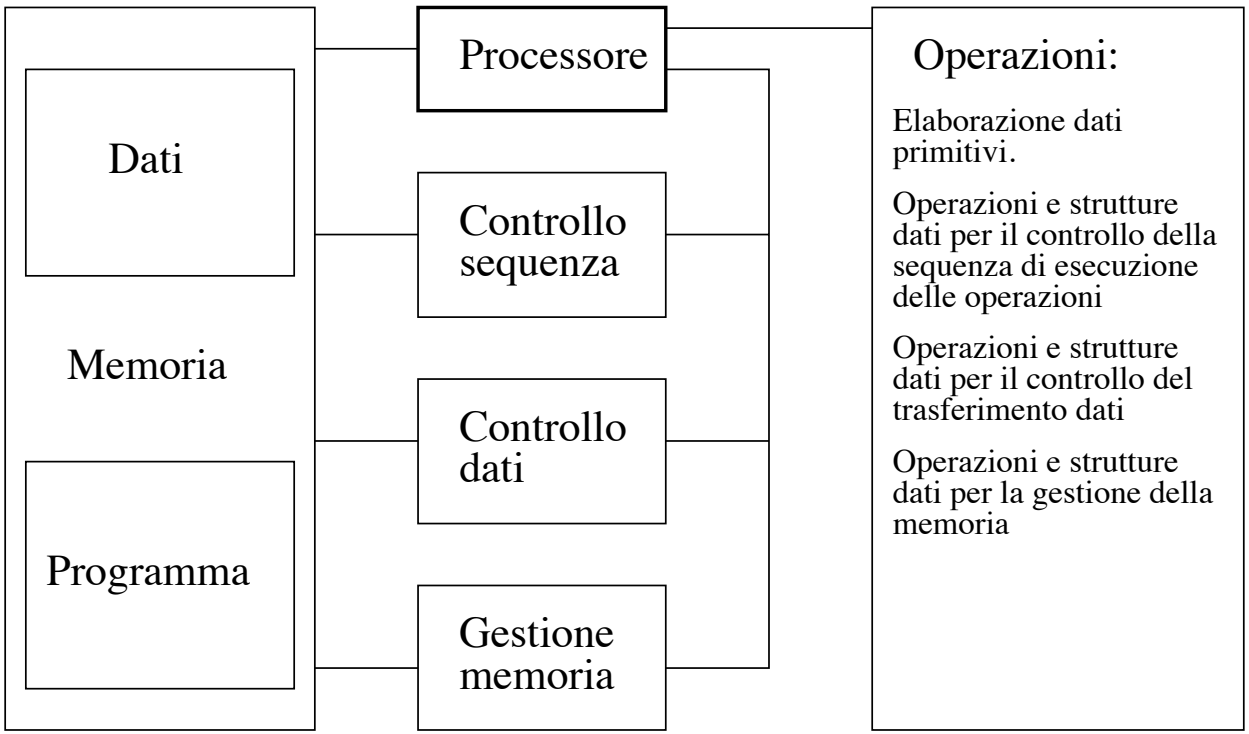


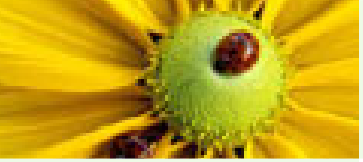
Definizione

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

Dato un linguaggio di programmazione L, una macchina astratta per L (in simboli, M_L) è un qualsiasi insieme di strutture dati e algoritmi che permettano di memorizzare ed eseguire programmi scritti in L.

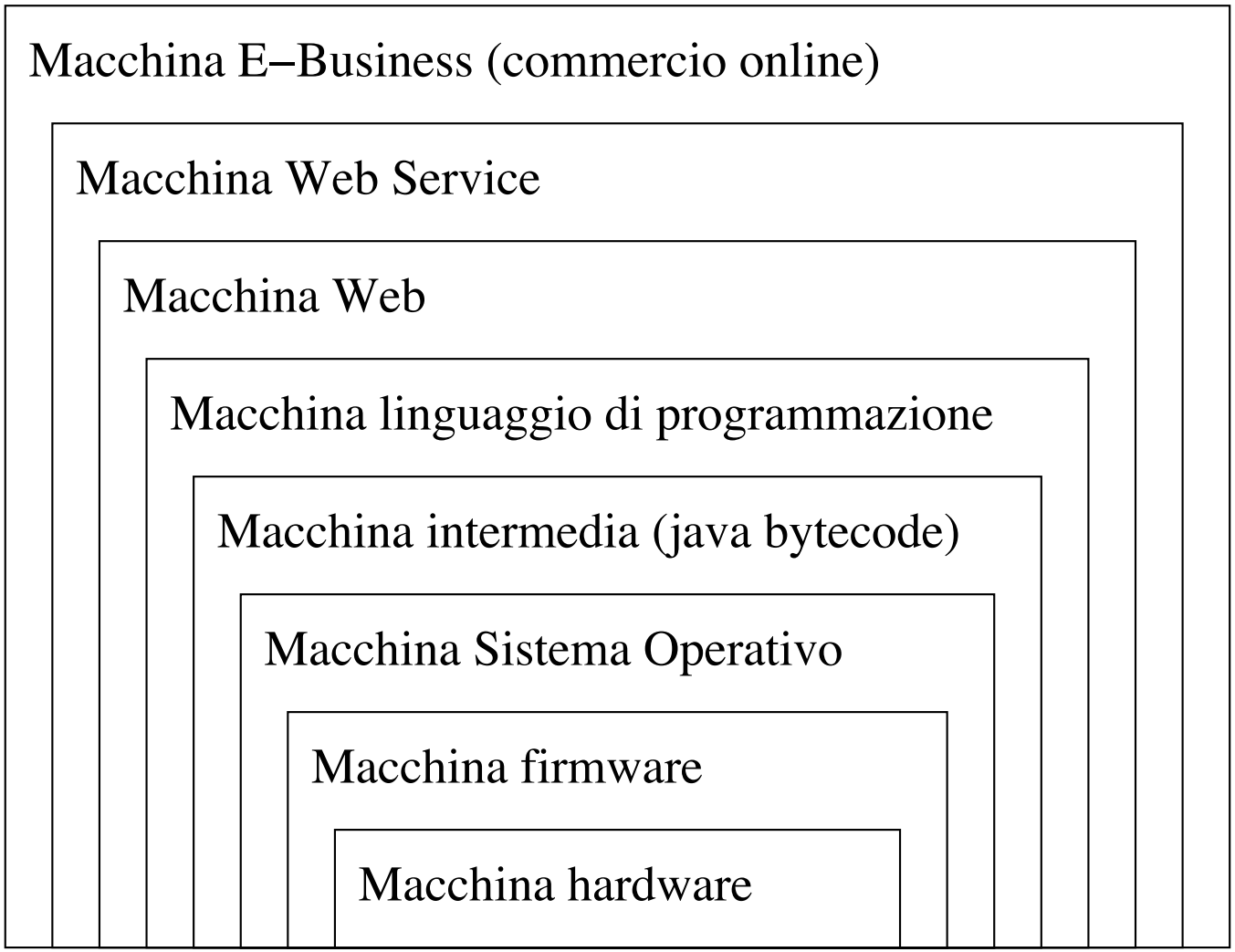
La struttura di una macchina astratta è (essenzialmente memoria e processore):

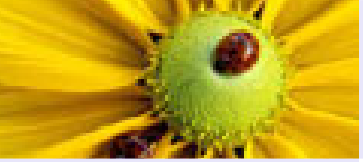




Esempio

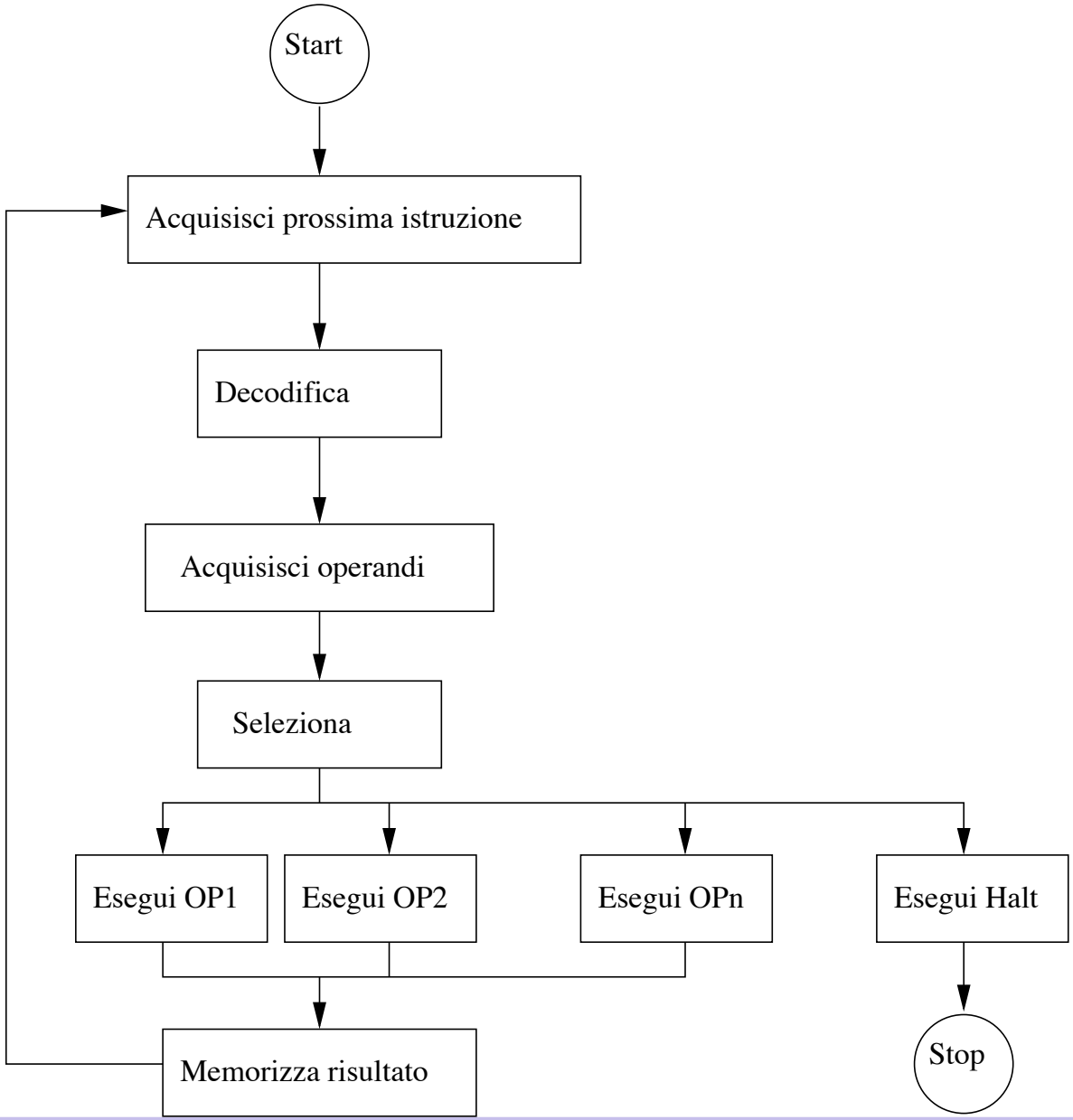
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

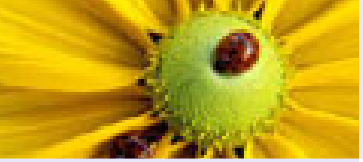




Processore della macchina astratta

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

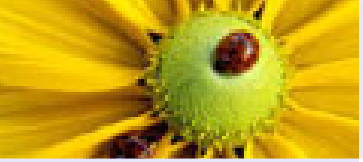




Esempi di macchine astratte

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

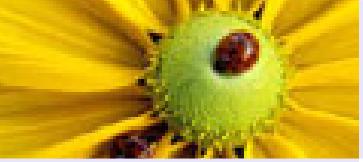
Linguaggio	Macchina astratta
C	gcc + OS + HW
Java	javac + JVM + OS + HW
C#	csc + CLR + OS + HW
Python	python + OS + HW



Tecnologie di realizzazione di macchina astratta

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

Hardware: Si era pensato per Lisp e Prolog

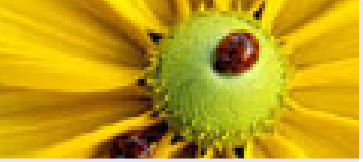


Tecnologie di realizzazione di macchina astratta

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

Hardware: Si era pensato per Lisp e Prolog

Firmware: Soluzione più flessibile e semplice / economicità di progetto



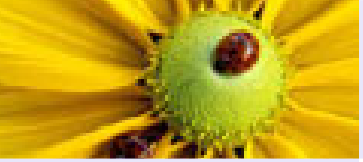
Tecnologie di realizzazione di macchina astratta

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

Hardware: Si era pensato per Lisp e Prolog

Firmware: Soluzione più flessibile e semplice / economicità di progetto

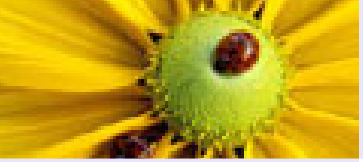
Software: Ad es. Java Virtual Machine, o Warren Abstract Machine (Prolog)



Traduttori Linguaggio → Macchina astratta

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

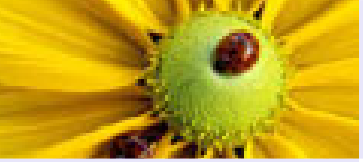
- **Interpreti**: traducono ed eseguono un costrutto alla volta.
  PRO: debug - fase di sviluppo: interazione più snella.



Traduttori Linguaggio → Macchina astratta

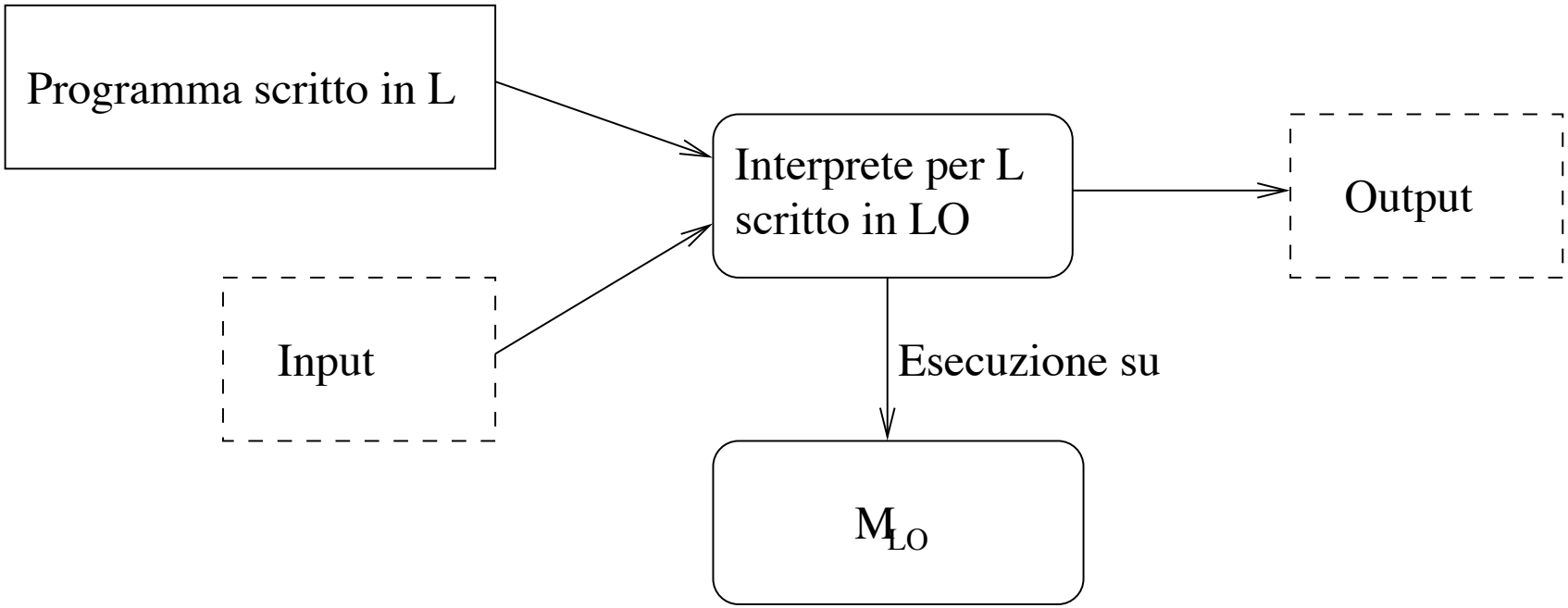
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

- **Interpreti**: traducono ed eseguono un costrutto alla volta.
  PRO: debug - fase di sviluppo: interazione più snella.
- **Compilatori**: prima traducono l'intero programma; poi la traduzione può essere eseguita (anche più volte).
  PRO: velocità di esecuzione finale - fase di rilascio.
  PRO: più controlli e in anticipo

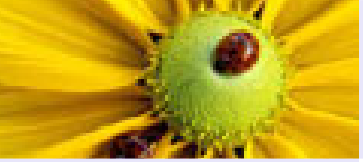


Interpretazione pura

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

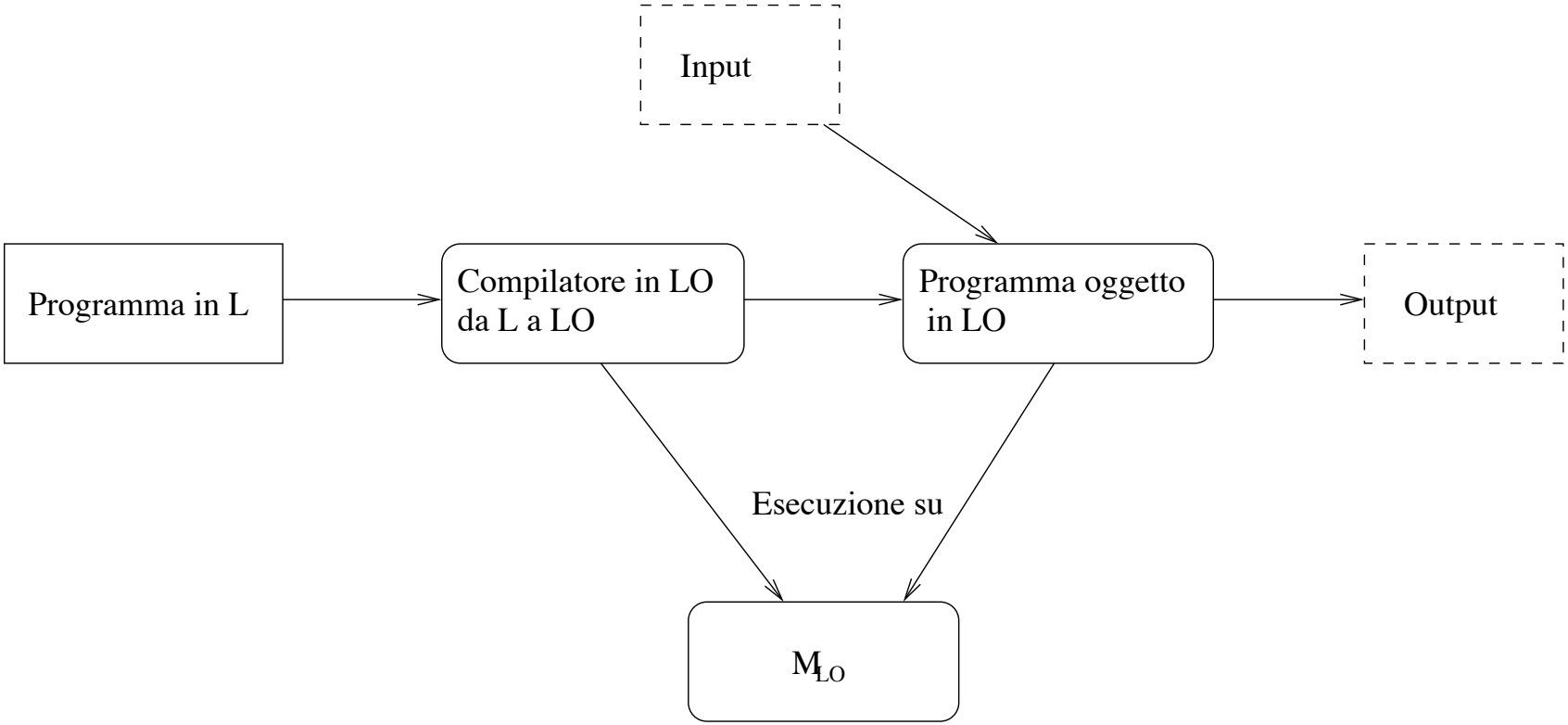


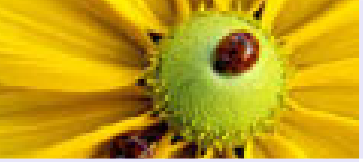
E' il caso dei linguaggi di scripting: bash, awk, Python, etc.



Compilazione pura (caso semplice)

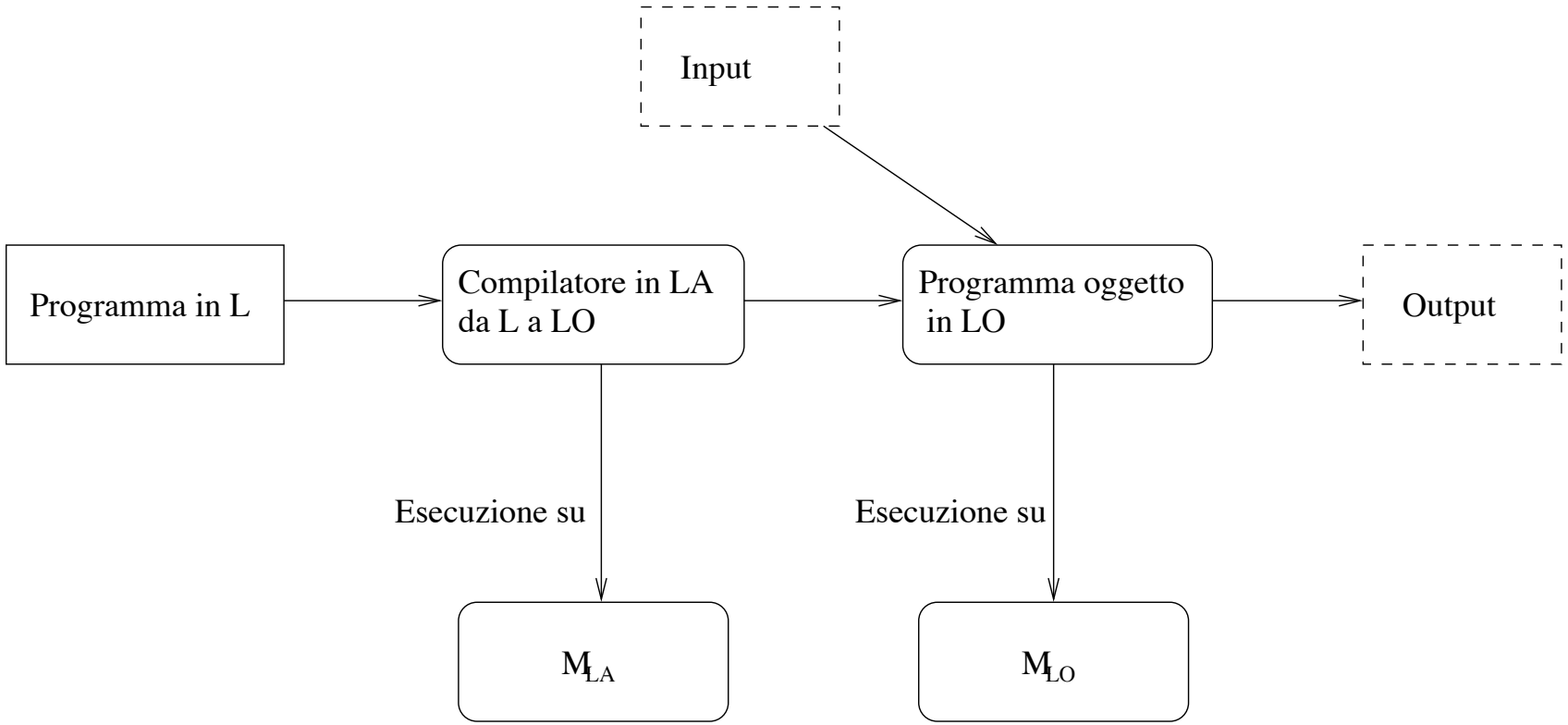
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

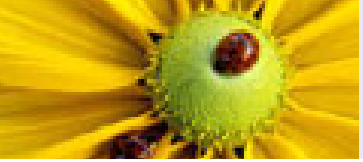




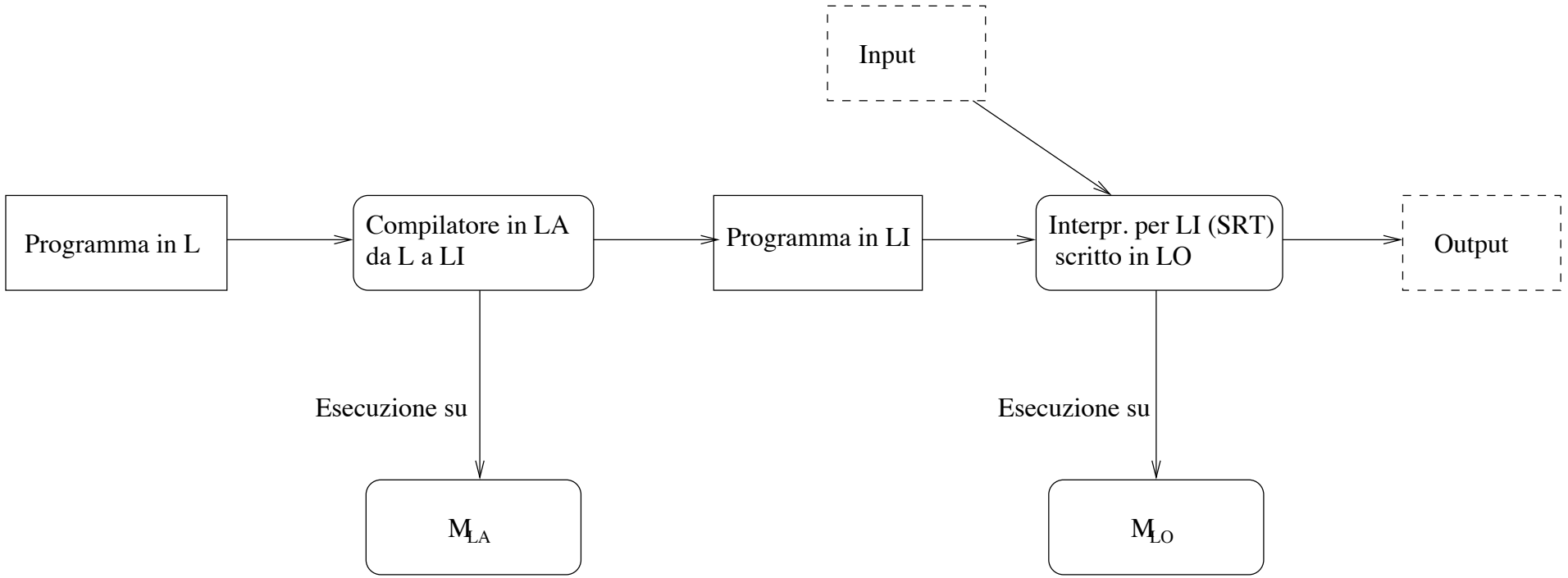
Compilazione pura (caso più generale)

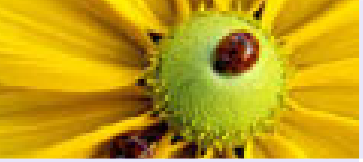
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali





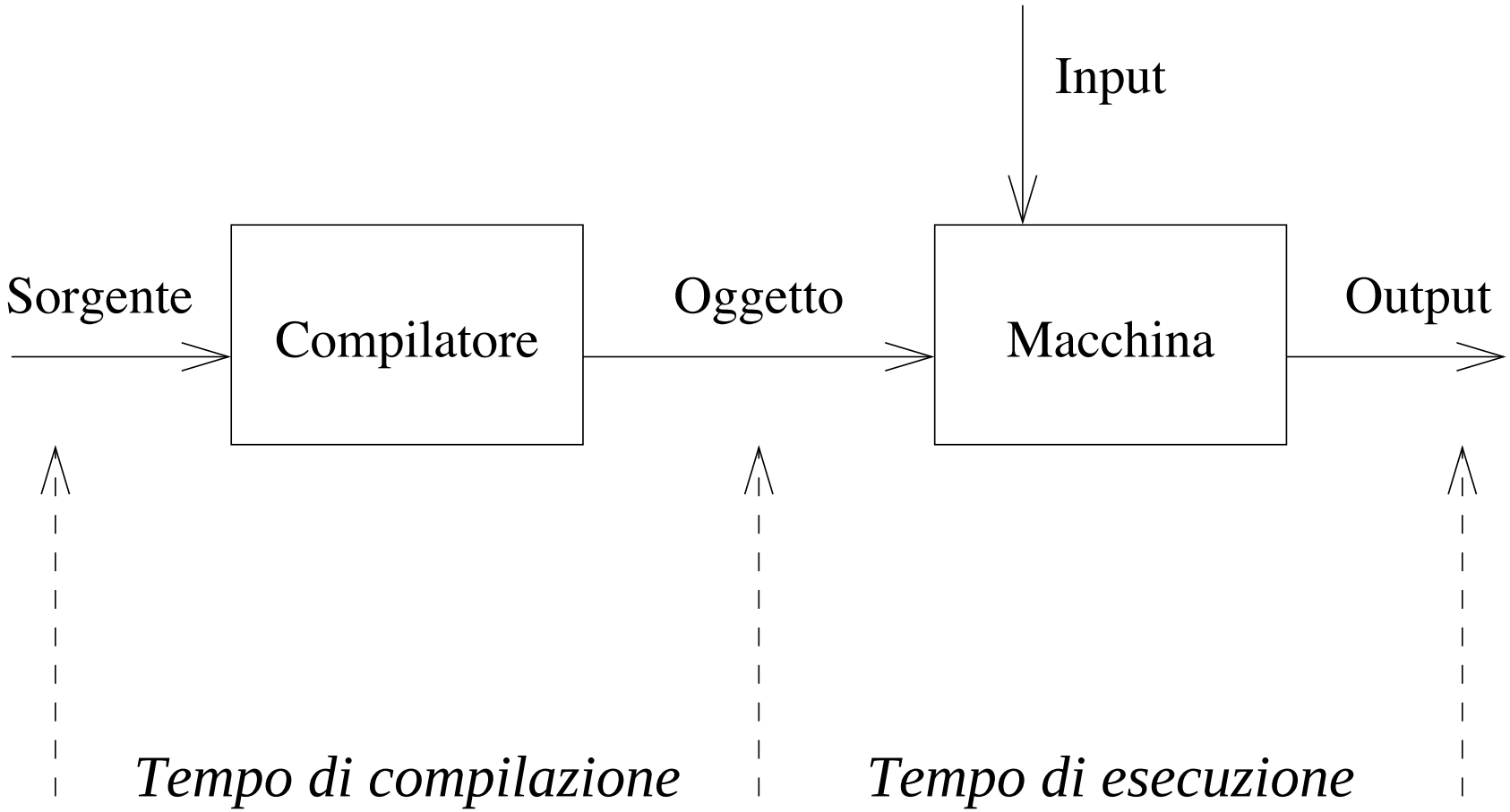
Compilazione per macchina intermedia

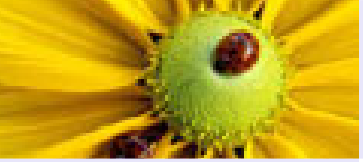




Compilazione ed esecuzione

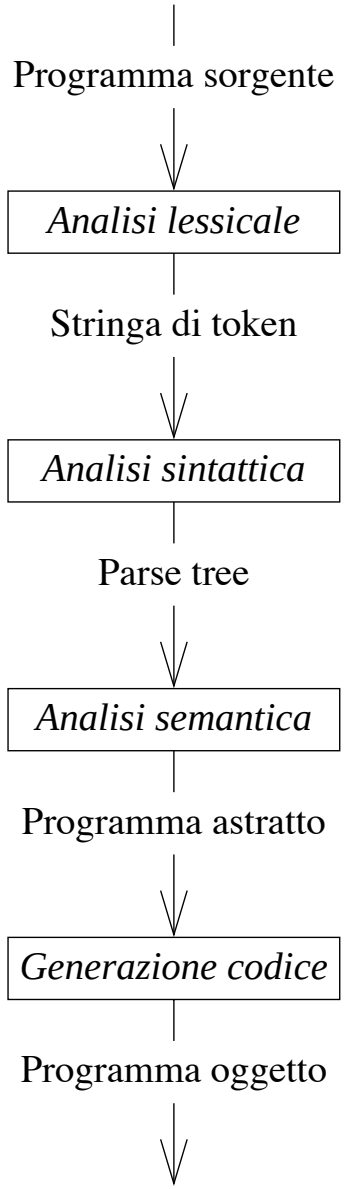
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

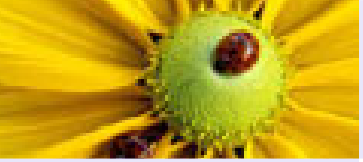




Compilatore

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

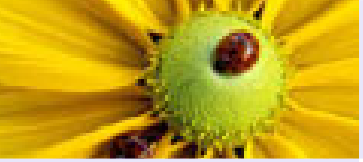




Componenti di un LP

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

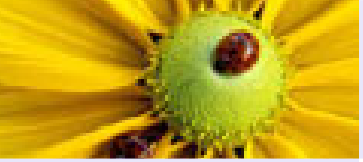
- Grammatica
  - *Come si scrive un programma ben formato (cioè, eseguibile)?*
  - Anche detta *sintassi*



Componenti di un LP

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

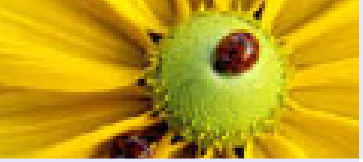
- Grammatica
  - *Come si scrive un programma ben formato (cioè, eseguibile)?*
  - Anche detta *sintassi*
- Semantica
  - *Qual è il significato di ciascuna istruzione/costrutto?*



Componenti di un LP

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

- Grammatica
  - *Come si scrive un programma ben formato (cioè, eseguibile)?*
  - Anche detta *sintassi*
- Semantica
  - *Qual è il significato di ciascuna istruzione/costrutto?*
- Pragmatica
  - *Come si usa efficacemente il linguaggio?* (Programmazione & Ingegneria del Software)

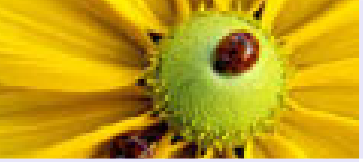


Componenti di un LP

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

- Grammatica
  - *Come si scrive un programma ben formato (cioè, eseguibile)?*
  - Anche detta *sintassi*
- Semantica
  - *Qual è il significato di ciascuna istruzione/costrutto?*
- Pragmatica
  - *Come si usa efficacemente il linguaggio?* (Programmazione & Ingegneria del Software)
- Implementazione
  - *Come si realizza una macchina astratta per questo linguaggio?* (Costruzione di compilatori)

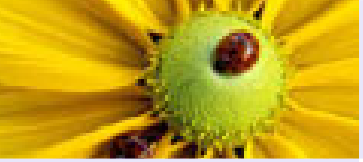
In questo corso, ci concentreremo su grammatica e semantica, con accenni all'implementazione



Proprietà dei linguaggi

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

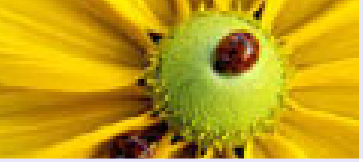
- Semplicità – (concisione) VS (leggibilità)
  - Sintattica: unica rappresentabilità di ogni concetto.
  - Semantica: minimo numero di concetti e strutture.



Proprietà dei linguaggi

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

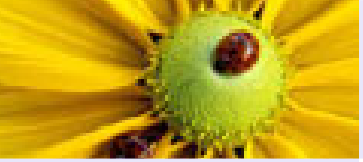
- Semplicità – (concisione) VS (leggibilità)
  - Sintattica: unica rappresentabilità di ogni concetto.
  - Semantica: minimo numero di concetti e strutture.
- Astrazione – (incapsulare e dividere)
  - Dati: nascondere i dettagli (incapsulamento, information hiding)
  - Funzioni e procedure: dividere il programma
  - Moduli, pacchetti, etc.: favorire la modularità



Proprietà dei linguaggi

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

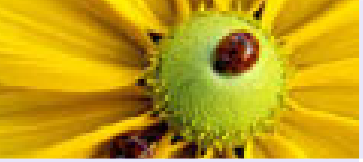
- Semplicità – (concisione) VS (leggibilità)
  - Sintattica: unica rappresentabilità di ogni concetto.
  - Semantica: minimo numero di concetti e strutture.
- Astrazione – (incapsulare e dividere)
  - Dati: nascondere i dettagli (incapsulamento, information hiding)
  - Funzioni e procedure: dividere il programma
  - Moduli, pacchetti, etc.: favorire la modularità
- Espressività – (facilità di rappresentazione di oggetti) VS (semplicità)



Proprietà dei linguaggi

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

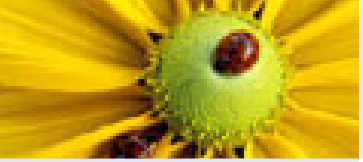
- Semplicità – (concisione) VS (leggibilità)
  - Sintattica: unica rappresentabilità di ogni concetto.
  - Semantica: minimo numero di concetti e strutture.
- Astrazione – (incapsulare e dividere)
  - Dati: nascondere i dettagli (incapsulamento, information hiding)
  - Funzioni e procedure: dividere il programma
  - Moduli, pacchetti, etc.: favorire la modularità
- Espressività – (facilità di rappresentazione di oggetti) VS (semplicità)
- Ortogonalità – (poche eccezioni alle regole del linguaggio)



Proprietà dei linguaggi

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

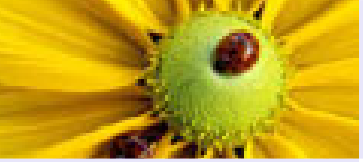
- Semplicità – (concisione) VS (leggibilità)
  - Sintattica: unica rappresentabilità di ogni concetto.
  - Semantica: minimo numero di concetti e strutture.
- Astrazione – (incapsulare e dividere)
  - Dati: nascondere i dettagli (incapsulamento, information hiding)
  - Funzioni e procedure: dividere il programma
  - Moduli, pacchetti, etc.: favorire la modularità
- Espressività – (facilità di rappresentazione di oggetti) VS (semplicità)
- Ortogonalità – (poche eccezioni alle regole del linguaggio)
- Portabilità



Criteri di scelta del linguaggio

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

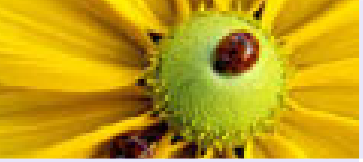
- Disponibilità di interpreti/compilatori



Criteri di scelta del linguaggio

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

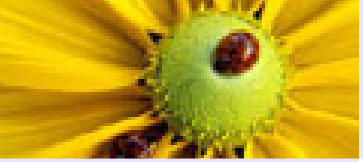
- Disponibilità di interpreti/compilatori
- Esistenza di standard di portabilità



Criteri di scelta del linguaggio

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

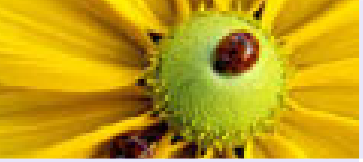
- Disponibilità di interpreti/compilatori
- Esistenza di standard di portabilità
- Interoperabilità con altri sistemi e linguaggi



Criteri di scelta del linguaggio

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

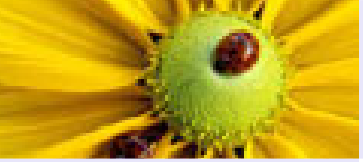
- Disponibilità di interpreti/compilatori
- Esistenza di standard di portabilità
- Interoperabilità con altri sistemi e linguaggi
- Esistenza di librerie specifiche



Criteri di scelta del linguaggio

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

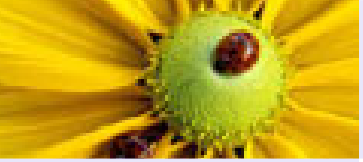
- Disponibilità di interpreti/compilatori
- Esistenza di standard di portabilità
- Interoperabilità con altri sistemi e linguaggi
- Esistenza di librerie specifiche
- Maggiore conoscenza da parte del programmatore



Criteri di scelta del linguaggio

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

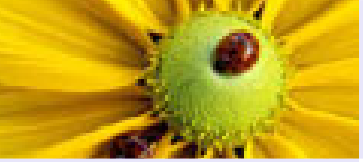
- Disponibilità di interpreti/compilatori
- Esistenza di standard di portabilità
- Interoperabilità con altri sistemi e linguaggi
- Esistenza di librerie specifiche
- Maggiore conoscenza da parte del programmatore
- Comodità dell'ambiente di programmazione (IDE)
- Sintassi aderente al problema



Criteri di scelta del linguaggio

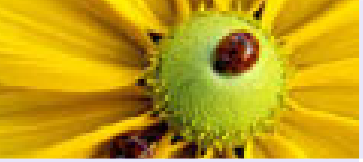
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Definizione
Esempio
Processore M.A.
Esempi M.A.
Realizzazione M.A.
Traduttori
Linguaggio → Macchina astratta
Interpretazione pura
Compilazione 1
Compilazione 2
Compilazione 3
Compilazione ed . . .
Compilatore
Componenti di un LP
Proprietà dei . . .
Criteri di scelta . . .
Paradigmi computazionali

- Disponibilità di interpreti/compilatori
- Esistenza di standard di portabilità
- Interoperabilità con altri sistemi e linguaggi
- Esistenza di librerie specifiche
- Maggiore conoscenza da parte del programmatore
- Comodità dell'ambiente di programmazione (IDE)
- Sintassi aderente al problema



Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

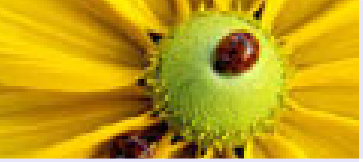
Paradigmi computazionali



Paradigmi

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

Imperativo: Un programma specifica sequenze di modifiche da apportare allo *stato della macchina* (memoria).

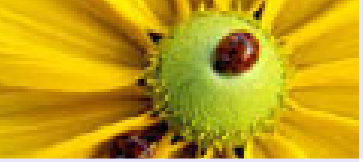


Paradigmi

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

Imperativo: Un programma specifica sequenze di modifiche da apportare allo *stato della macchina* (memoria).

Funzionale: Il programma e le sue componenti sono *funzioni*. Esecuzione come valutazione di funzioni.



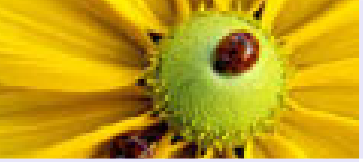
Paradigmi

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0 Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

Imperativo: Un programma specifica sequenze di modifiche da apportare allo *stato della macchina* (memoria).

Funzionale: Il programma e le sue componenti sono *funzioni*. Esecuzione come valutazione di funzioni.

Logico: Programma come descrizione logica di un problema. Esecuzione analoga a processi di dimostrazione di teoremi.



Paradigmi

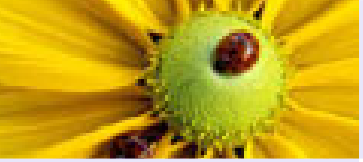
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

Imperativo: Un programma specifica sequenze di modifiche da apportare allo *stato della macchina* (memoria).

Funzionale: Il programma e le sue componenti sono *funzioni*. Esecuzione come valutazione di funzioni.

Logico: Programma come descrizione logica di un problema. Esecuzione analoga a processi di dimostrazione di teoremi.

Orientato ad oggetti: Programma costituito da oggetti che scambiano messaggi.

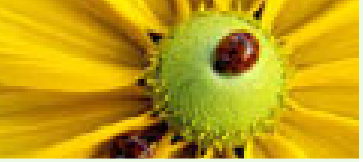


Paradigmi

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

Imperativo: Un programma specifica sequenze di modifiche da apportare allo *stato della macchina* (memoria).
Funzionale: Il programma e le sue componenti sono *funzioni*. Esecuzione come valutazione di funzioni.
Logico: Programma come descrizione logica di un problema. Esecuzione analoga a processi di dimostrazione di teoremi.
Orientato ad oggetti: Programma costituito da oggetti che scambiano messaggi.
Parallelo: Programmi che descrivono entità distribuite che sono eseguite contemporaneamente ed in modo asincrono.

Gli ultimi due sono ortogonali rispetto ai primi tre...



Esempio 1.0 Imperativo vs. Funzionale

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0 Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

function definition

```
function factI (n)
  local accumulator = 1
  for i = 1,n do
    accum = accumulator*i
  end
  return accum
end
```

trace of execution

```
factI(4):
          accumulator = 1
i = 1     accumulator = 1 * 1
i = 2     accumulator = 1 * 2
i = 3     accumulator = 2 * 3
i = 4     accumulator = 6 * 4
return 24
```

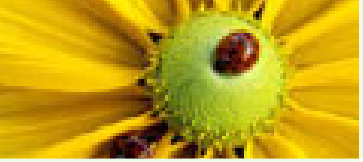
function definition

```
function factR (n)
  if n == 1 then
    return 1
  else
    return n*factR(n-1)
  end
end
```

trace of execution

```
factR(4) =
4 * factR(3) =
      3 * factR(2) =
            2 * factR(1) =
                        1
                        1
                  1
            2
      6
24
```

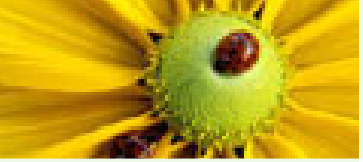
Notare eliminazione assegnamenti: n rimpiazza i, ricorsione invece di cicli



Esempio 1.1

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

Vogliamo scrivere in un linguaggio imperativo, funzionale e logico la funzione `member(X, L)` che decida se l'elemento `X` appartiene alla lista `L`.



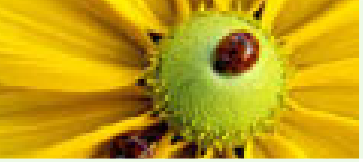
Esempio 1.1

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0 Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

Vogliamo scrivere in un linguaggio imperativo, funzionale e logico la funzione `member(X, L)` che decida se l'elemento `X` appartiene alla lista `L`.

Es.:

```
member(2, [1, 2, 3]) = true;
member(4, [1, 2, 3]) = false.
```



Esempio 1.1

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0 Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

Vogliamo scrivere in un linguaggio imperativo, funzionale e logico la funzione `member(X, L)` che decida se l'elemento `X` appartiene alla lista `L`.

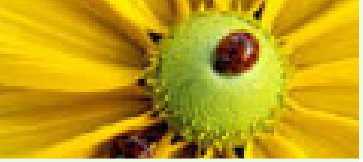
Es.:

`member(2, [1, 2, 3]) = true;`

`member(4, [1, 2, 3]) = false.`

A questo scopo, si suppongano già esistenti le funzioni:

- `empty(L)`, che restituisce `true` se `L` è vuota, altrimenti `false`.



Esempio 1.1

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0 Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

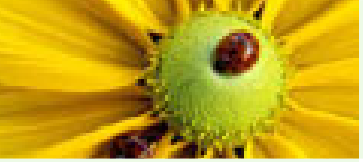
Vogliamo scrivere in un linguaggio imperativo, funzionale e logico la funzione `member(X, L)` che decida se l'elemento `X` appartiene alla lista `L`.

Es.:

```
member(2, [1, 2, 3]) = true;
member(4, [1, 2, 3]) = false.
```

A questo scopo, si suppongano già esistenti le funzioni:

- `empty(L)`, che restituisce `true` se `L` è vuota, altrimenti `false`.
- `first(L)`, che restituisce il primo elemento della lista `L`.
  Es.: `first([1, 2, 3]) = 1`.



Esempio 1.1

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0 Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

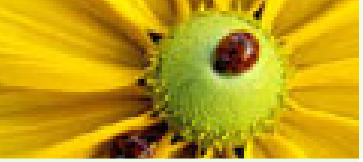
Vogliamo scrivere in un linguaggio imperativo, funzionale e logico la funzione `member(X, L)` che decida se l'elemento `X` appartiene alla lista `L`.

Es.:

```
member(2, [1, 2, 3]) = true;
member(4, [1, 2, 3]) = false.
```

A questo scopo, si suppongano già esistenti le funzioni:

- `empty(L)`, che restituisce `true` se `L` è vuota, altrimenti `false`.
- `first(L)`, che restituisce il primo elemento della lista `L`.
  Es.: `first([1, 2, 3]) = 1`.
- `rest(L)`, che restituisce una sottolista ottenuta rimuovendo il primo elemento di `L`.
  Es.: `rest([1, 2, 3]) = [2, 3]`.

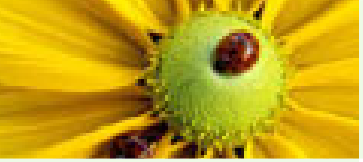


Esempio 1.2

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

La funzione `member` nel paradigma imperativo (pseudo-codice):

```
procedure member(X,L)
  local L1 = L
  while not empty(L1) and X != first(L1)
    do L1 = rest(L1)
  return not empty(L1)
```



Esempio 1.2

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

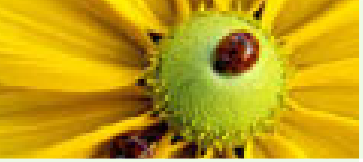
La funzione `member` nel paradigma imperativo (pseudo-codice):

```
procedure member(X,L)
  local L1 = L
  while not empty(L1) and X != first(L1)
    do L1 = rest(L1)
  return not empty(L1)
```

In C (un concreto linguaggio imperativo):

```
bool member(int X, Lista L) {
  Lista L1 = L;
  while( ! empty(L1) && X != first(L1))
    L1 = rest(L1);
  return ! empty(L1);
}
```

NB: nessuna differenza strutturale, solo dettagli sintattici

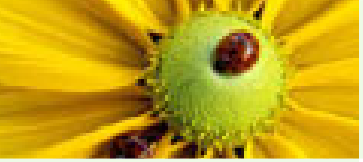


Esempio 2

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

La funzione `member` nel paradigma funzionale:

```
fun member X L =
   if (empty L) then false
   else if  X = (first L) then true
   else member X (rest L)
```



Esempio 2

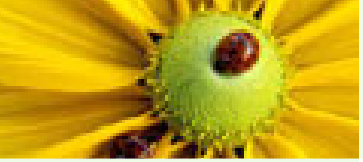
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

La funzione `member` nel paradigma funzionale:

```
fun member X L =
   if (empty L) then false
   else if  X = (first L) then true
   else member X (rest L)
```

Nella versione funzionale pura non ci sono variabili, nè assegnazioni. Quindi non si possono usare cicli e bisogna rimpiazzarli con la ricorsione.
La sintassi Lisp e Scheme sarebbe un po' particolare:

```
(defun member (x l)
  (cond ((null l) nil)
        ((equal x (first l)) T)
        (T (member x (rest l)))))
```



Esempio 2

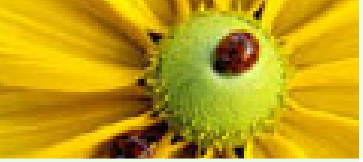
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

La funzione `member` nel paradigma funzionale:

```
fun member X L =
   if (empty L) then false
   else if  X = (first L) then true
   else member X (rest L)
```

Anche il C si potrebbe usare in stile funzionale se evitassimo di usare i costrutti imperativi:

```
bool member(int X, Lista L) {
   return (empty(L))?
          false :
          (
             (X == first(L))?
             true :
             member(X, rest(L))
          );
}
```



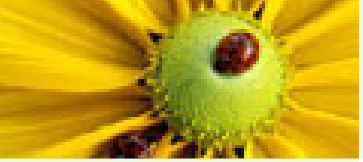
Esempio 3

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

La funzione `member` nel paradigma logico (prolog):

```
member(X, [X|L]).
member(X, [Y|L]) :- member(X, L).
```

- `[X|L]` è un *pattern* che denota una lista in cui X è la testa e L è il resto
- `member` è un *predicato*
- I parametri formali dei predicati possono essere pattern
- I programmi consistono di definizioni di predicati



Esempio 3

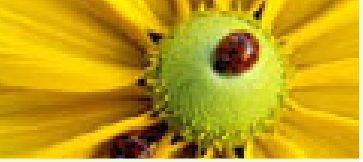
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

La funzione `member` nel paradigma logico (prolog):

```
member(X, [X|L]).
member(X, [Y|L]) :- member(X, L).
```

Esecuzione:

- member(2, [1,2,3]) restituisce *yes* (true)
- member(0, [1,2,3]) restituisce *no* (false)



Esempio 3

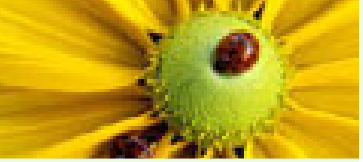
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0 Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

La funzione `member` nel paradigma logico (prolog):

```
member(X, [X|L]).
member(X, [Y|L]) :- member(X, L).
```

Esecuzione:

- member(2, [1,2,3]) restituisce *yes* (true)
- member(0, [1,2,3]) restituisce *no* (false)
- Query con variabili: member(X,[1,2,3]) restituisce
  - `X=1`



Esempio 3

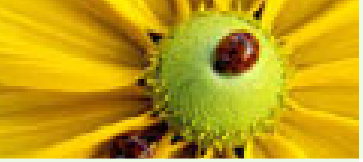
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

La funzione `member` nel paradigma logico (prolog):

```
member(X, [X|L]).
member(X, [Y|L]) :- member(X, L).
```

Esecuzione:

- member(2, [1,2,3]) restituisce *yes* (true)
- member(0, [1,2,3]) restituisce *no* (false)
- Query con variabili: member(X,[1,2,3]) restituisce
  - `X=1; X=2`



Esempio 3

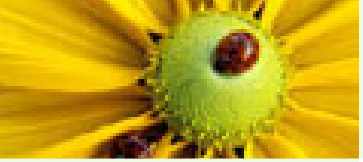
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

La funzione `member` nel paradigma logico (prolog):

```
member(X, [X|L]).
member(X, [Y|L]) :- member(X, L).
```

Esecuzione:

- member(2, [1,2,3]) restituisce *yes* (true)
- member(0, [1,2,3]) restituisce *no* (false)
- Query con variabili: member(X,[1,2,3]) restituisce
  - `X=1; X=2; X=3`



Esempio 3

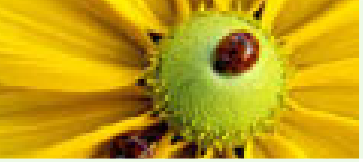
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

La funzione `member` nel paradigma logico (prolog):

```
member(X, [X|L]).
member(X, [Y|L]) :- member(X, L).
```

Esecuzione:

- member(2, [1,2,3]) restituisce *yes* (true)
- member(0, [1,2,3]) restituisce *no* (false)
- Query con variabili: member(X,[1,2,3]) restituisce
  - `X=1; X=2; X=3; no` (si comporta come un generatore)



Esempio 3

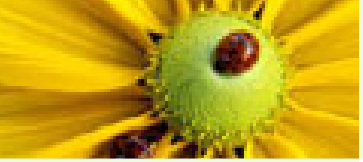
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0 Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

La funzione `member` nel paradigma logico (prolog):

```
member(X, [X|L]).
member(X, [Y|L]) :- member(X, L).
```

Esecuzione:

- member(2, [1,2,3]) restituisce *yes* (true)
- member(0, [1,2,3]) restituisce *no* (false)
- Query con variabili: member(X,[1,2,3]) restituisce
  - `X=1; X=2; X=3; no` (si comporta come un generatore)
- Risposte con variabili: member(1,L) restituisce
  - `L=[1|`L_0`]`



Esempio 3

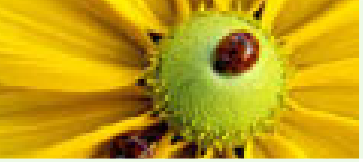
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0 Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

La funzione `member` nel paradigma logico (prolog):

```
member(X, [X|L]).
member(X, [Y|L]) :- member(X, L).
```

Esecuzione:

- member(2, [1,2,3]) restituisce *yes* (true)
- member(0, [1,2,3]) restituisce *no* (false)
- Query con variabili: member(X,[1,2,3]) restituisce
  - `X=1; X=2; X=3; no` (si comporta come un generatore)
- Risposte con variabili: member(1,L) restituisce
  - `L=[1|`L_0`]; L=[`Y_0`,1|`L_1`]`



Esempio 3

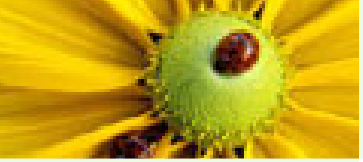
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0 Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

La funzione `member` nel paradigma logico (prolog):

```
member(X, [X|L]).
member(X, [Y|L]) :- member(X, L).
```

Esecuzione:

- member(2, [1,2,3]) restituisce *yes* (true)
- member(0, [1,2,3]) restituisce *no* (false)
- Query con variabili: member(X,[1,2,3]) restituisce
  - `X=1; X=2; X=3; no` (si comporta come un generatore)
- Risposte con variabili: member(1,L) restituisce
  - `L=[1|`L_0`]; L=[`Y_0`,1|`L_1`]; L=[`Y_0`,`Y_1`,1|`L_2`] ...`



Esempio 3

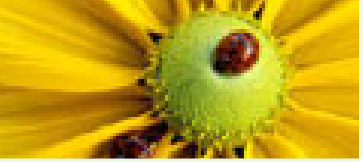
Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

La funzione `member` nel paradigma logico (prolog):

```
member(X, [X|L]).
member(X, [Y|L]) :- member(X, L).
```

Esecuzione:

- member(2, [1,2,3]) restituisce *yes* (true)
- member(0, [1,2,3]) restituisce *no* (false)
- Query con variabili: member(X,[1,2,3]) restituisce
  - `X=1; X=2; X=3; no` (si comporta come un generatore)
- Risposte con variabili: member(1,L) restituisce
  - `L=[1|L0]; L=[Y0,1|L1]; L=[Y0,Y1,1|L2] ...`
- *Invertibilità*: nessuna distinzione tra input e output. Un solo predicato (programma), molte funzioni.



Conclusioni

Introduzione al corso
Linguaggi (di programmazione)
Macchine astratte
Paradigmi computazionali
Paradigmi
Esempio 1.0
Imperativo vs. Funzionale
Esempio 1.1
Esempio 1.2
Esempio 2
Esempio 2
Esempio 3
Esempio 3
Conclusioni

- Il paradigma di appartenenza può influenzare *radicalmente* il modo in cui si risolve il problema
  - In linguaggi dello stesso paradigma, lo stesso problema ha soluzioni strutturalmente identiche
  - Imparato a risolvere un problema in un linguaggio, lo si sa risolvere in tutti i linguaggi dello stesso paradigma
- Il paradigma non è l'unico aspetto determinante. Altri esempi di aspetti importanti:
  - Il sistema di tipi supportato
  - Eventuale supporto alle eccezioni
  - Modello di concorrenza e sincronizzazione
  - Presenza di garbage collection
  - ...